

ual:

Coding Three

Creative Robotics

Week 3: 16 October 2025

Today's Agenda

- Lecture 1: Advanced OpenCV Techniques
 - Edge Detection: Finding boundaries in images
 - Orientation Detection: Understanding object angles
 - Kernels and Convolution Operations
- Lecture 2: What is Machine Learning?
 - Types of ML
 - Introduction to ML frameworks
- Lab time
- In-Class Assignment 1 (part of Element 1 hand-in)



Advanced OpenCV:

Let's learn how computers “see” edges and angles



Quick Recap

■ Week 1:

- OpenCV basics: reading images, color spaces
- Basic image operations

■ Week 2:

- Erosion and Dilation (morphological operations)
- Blob Detection
- Background Subtraction
- Contours

Quick! Which is Dilation?



Quick! Which is Dilation?



Original



Dilation



Erosion

Today:

- Today:
 - More sophisticated image analysis techniques
 - Bridge to machine learning concepts

What is an Edge?

- An edge is a boundary in an image where:
 - Pixel brightness changes sharply
 - Object boundaries exist
 - Texture changes
 - Color transitions occur
- Like drawing an outline of objects in your image
- Edges are among the most important features in image: They tell us the underlying structure

Edge Detection

- Edge detection is an image-processing technique that is used to identify the boundaries (edges) of objects or regions within an image
 - It finds places where pixel values change dramatically.
- Edges are found at:
 - Boundaries between objects
 - Sudden changes in brightness
 - Transitions in color
 - Changes in texture



Why Do We Need Edge Detection?

- Simplify images: Reduce data while keeping important structures
- Find object shapes: Identify outlines and boundaries
- Detect motion: Changes in edge indicate movement
- Recognize patterns: Edges define object characteristics

Edge Detection



Real-world example:

Self-driving cars use edge detection to see lane markings on roads

Image credit (also a great tutorial): <https://medium.com/neurosapiens/edge-detection-65d57c1af118>

Canny Edge Detection

Canny Edge Detection

Image credit (also a great tutorial): <https://medium.com/data-science/canny-edge-detection-step-by-step-in-python-computer-vision-b49c3a2d8123>

- A very popular edge detection method
- It's like having a smart filter that finds edges while reducing noise (unwanted details)
- Multi-stage algorithm in 5 steps:
 1. Noise reduction
 2. Gradient calculation
 3. Non-maximum suppression
 4. Double threshold
 5. Edge tracking by hysteresis



Original image on the left — Processed image on the right

Canny Edge Detection: Step 1 Noise Reduction

- **What it does:** Smooths the image to remove random pixel variations (noise) that could be mistaken for edges
- **How:** by applying Gaussian blur, which we also saw in Week 2
- Here is the line #20 from your lab **a_contour_edge_detect_webcam.py**

```
img_blur = cv.GaussianBlur(frame, (3,3), 0)
```

- **(3, 3)** is the kernel size, the blur “brush”
- **0** means let OpenCV calculate the standard deviation automatically
- Large kernel = more blur = less noise but may lose fine details

Canny Edge Detection: Step 2 Gradient Calculation

- **What it does:** Finds the intensity (brightness) changes in the image.
- Where brightness changes rapidly = potential edge
- **How:** Canny finds edges by first smoothing the image, then measuring how brightness changes using simple filters (Sobel) to get the gradient magnitude and direction
 - We use the Sobel filters because they give a quick estimate of horizontal and vertical change, which lets Canny find where edges are and how they're oriented.
- This happens *inside* of the the **`cv.Canny()`** function, so you don't see this code directly. OpenCV does it for you

Canny Edge Detection: Step 3 Non-Maximum Suppression

- **What it does:** Thins out thick edges to make them one pixel wide
- **How:** Keeps only the brightest pixels along the edge direction, removes the rest
- This also happens *inside* of the the **cv.Canny()** function automatically. OpenCV handles this for you
- Think of it as drawing a clean line instead of a thick, messy marker stroke

Canny Edge Detection: Step 4 Double Threshold

- **What it does:** Decides which edges are “strong”, “weak” or “not an edge”
- **How:** Using the two threshold values (Line #23 in your lab)

```
edges = cv.Canny(image=img_blur, threshold1=100, threshold2=200)
```

- **threshold1=100 (lower):** Minimum edge strength
 - Pixels with gradient > 100 but < 200 are "weak edges" (maybe)
- **threshold2=200 (upper):** Strong edge threshold
 - Pixels with gradient > 200 are "strong edges" (definitely keep!)
- Pixels < 100 are discarded (not edges)

Canny Edge Detection: Step 5 Edge Tracking by Hysteresis

- Hysteresis lets Canny keep faint parts of a real edge (weak pixels) only when they link to a definite edge (strong pixels), which reduces spurious isolated responses. Think “follow the strong edges and keep the nearby weak trail.”
- **What it does:**
 - Connects weak edges to strong edges
 - Discards weak edges that stand alone
- **How:** If a weak edge pixel is connected to a strong edge pixel, keep it. Otherwise, discard it
- This also happens *inside* of the the **cv.Canny()** function automatically: it traces connected edges for you. Which will result clean, continuous edge lines for you

Canny Edge Detection: Step 5 Edge Tracking by Hysteresis

- Think of Canny's hysteresis as tracing a drawing:
 - Strong, clear strokes are definitely part of the contour
 - Faint strokes are kept only if they connect to those clear strokes and help form a continuous outline.
 - Isolated faint marks that don't join a strong stroke get ignored as noise
- Canny traces the bold lines first, then follows and keeps the faint pencil marks that help complete those bold outlines, but it throws away stray pencil dots that don't connect to anything.

Understanding Canny Thresholds

■ Think of the thresholds like setting **sensitivity on a motion detector**

■ **threshold1** (lower):

■ Minimum edge strength to consider

■ Lower value:

■ Detects more edges

■ More sensitive

■ More noise included

■ **threshold2** (upper):

■ Defines “strong” edges

■ Higher value:

■ Only strong edges

■ Less sensitive

■ Clean results

Canny Edge Detection: `a_contour_edge_detect_webcam.py`

■ What the code does:

- Capture webcam feed
- Applies Gaussian blur for noise reduction
- Performs Canny edge detection

■ What you will see:

- Window 1: original webcam feed
- Window 2: Edge detected output showing white edges on black background

Canny Edge Detection: `a_contour_edge_detect_webcam.py`

- Let's run it and see!

- Go to your Terminal

- **cd** to your file path (where you store the files for today's class)

- You want file path of the folder where your lab is in, not the actual lab file

- Activate your conda: **conda activate coding3**

- **python a_contour_edge_detect_webcam.py**

Canny Edge Detection: `a_contour_edge_detect_webcam.py`

■ Try:

- Hold up object with clear edges
- Move your hand around
- Observe how threshold values affect which edges are detected
 - Change `(3,3)` to `(7,7)`
 - Change `threshold1=50`
 - Change `threshold2=300`

Canny Edge Detection: `a_contour_edge_detect_webcam.py`

- Experiment with these values and observe:
 - Change `(3,3)` to `(7,7)`: more blur, smoother edges, less noise but less detail
 - Change `threshold1=50`: detects more edges (including noise), more sensitive
 - Change `threshold2=300`: only very strong edges remain, less sensitive, cleaner results
- The ratio between `threshold1` and `threshold2` is usually 1:2 or 1:3
- Experiment with these values for different images and see the differences in results
- Reference: <https://learnopencv.com/edge-detection-using-opencv/>

Sobel Edge Detection (Quick Explanation)

■ What it is

- Computes image gradients using two small filters (Sobel X and Sobel Y).
- Gives edge strength and orientation by combining those horizontal and vertical responses.

■ Why it's different from Canny

- Simpler: computes gradient magnitude/direction but does not do non-maximum suppression or hysteresis.
- Faster and lighter weight — good for quick or real-time use

Sobel Edge Detection (Quick Explanation)

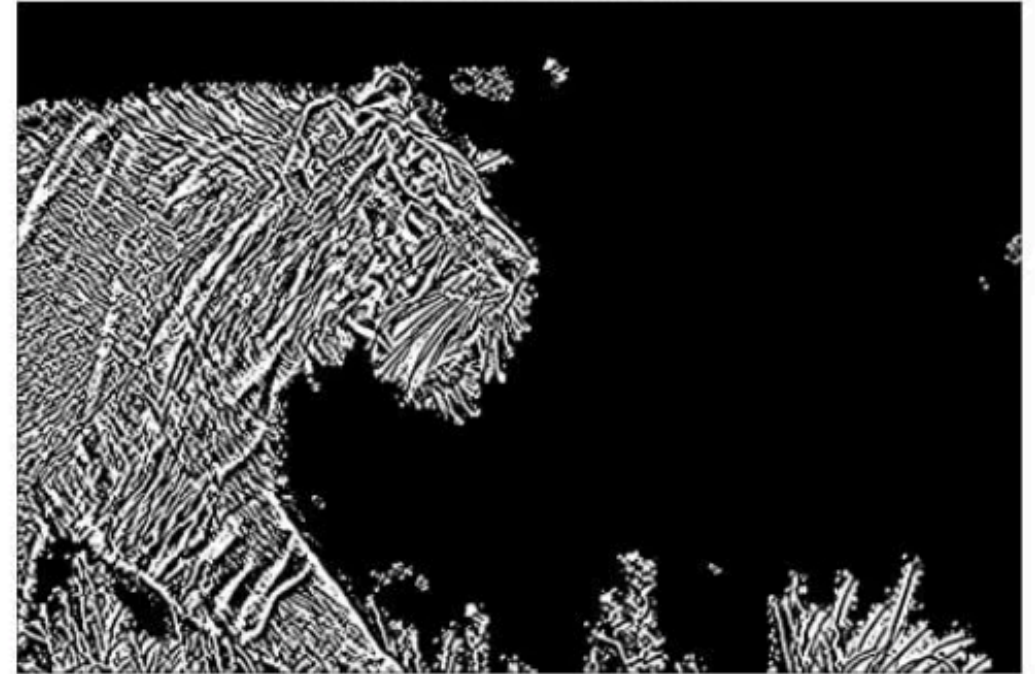
- When to use
 - If you need directional edge info (horizontal vs vertical)
 - For simple, fast edge detection or preprocessing (e.g., feature extraction), or when full Canny pipeline is unnecessary
- Canny uses Sobel (or similar) filters to compute gradients as one step of its pipeline, but “Sobel edge detection” usually refers to simply using those gradient maps (and thresholding) as the final edge output
- So Sobel alone is a simpler, standalone method, while Canny uses Sobel as an internal step plus extra processing (non-maximum suppression and hysteresis) for cleaner edges

Sobel Edge Detection (Quick Explanation)

Original



After Sobel Filter



Comparison of Original and Sobel Filter image

Image credit (also a great tutorial): <https://learnopencv.com/edge-detection-using-opencv/>

Better results with Canny Edge Detection



Final image after Canny Edge Detection

Image credit (also a great tutorial): <https://learnopencv.com/edge-detection-using-opencv/>

Orientation Detection

What is Orientation Detection?

- Orientation detection determines:
 - Which way an object is pointing
 - The rotation angle of an object
 - The principal axes of an object



What is Orientation Detection?

- Applications in robotics:
 - Object manipulation (pick and place)
 - Pose estimation
 - Understanding object rotation
 - Aligning robot gripper with object
- Example: A robot needs to know if a bottle is lying down or standing up before grasping it



Principal Component Analysis (PCA)

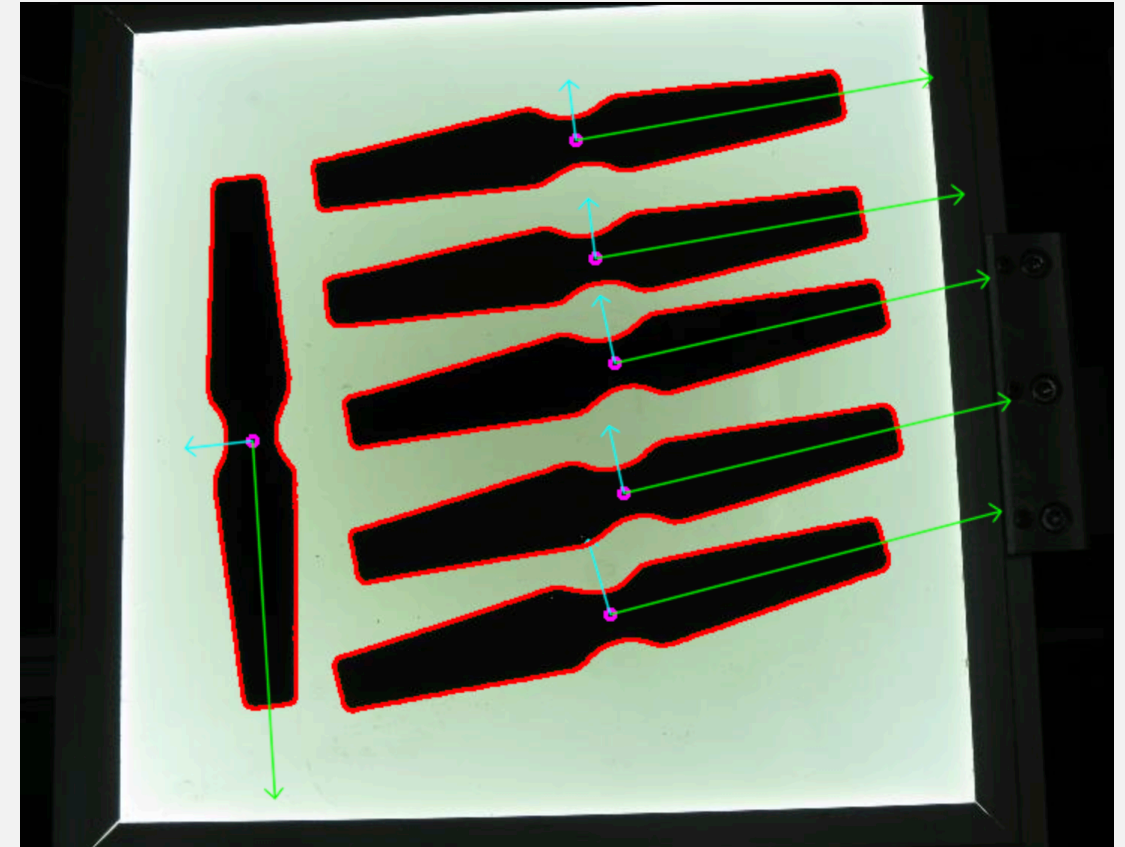
- PCA is a mathematical technique that:
 - Finds the direction along which data varies the most
 - Identifies the principal axes of an object
 - Calculates orientation angles
- From the OpenCV documentation: "PCA allows us to find the direction along which our data varies the most. The result of running PCA on a set of points consists of 2 vectors called eigenvectors which are the principal components of the data set."

Principal Component Analysis (PCA)

■ In simpler terms:

- PCA Finds the **longest** and **widest** directions of an object
- These directions tell us how the object is oriented
- Finds the orientation of the detected objects of interest and then visualizes the result by drawing the contours of the detected objects of interest, the center point, and the x-axis, y-axis regarding the extracted orientation

Image credit (also a great tutorial): https://docs.opencv.org/4.x/d1/dee/tutorial_introduction_to_pca.html

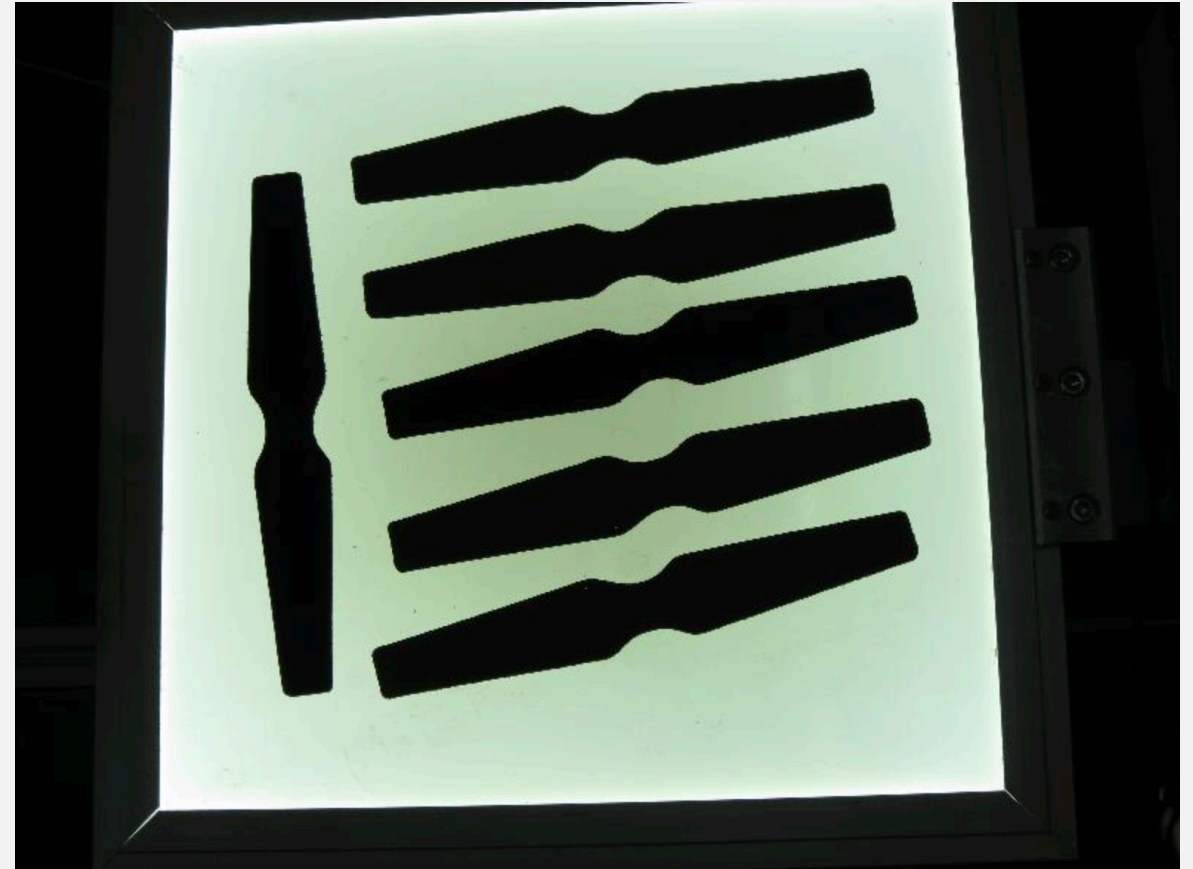


How PCA Finds Orientation

■ The process:

1. Find all points on the object's contour
 2. Calculate the center point (centroid)
 3. Use PCA to find two principal axes:
 1. Major axis (longest direction)
 2. Minor axis (perpendicular to major axis)
 4. Calculate the angle of the major axis
- The angle tells us the object's orientation.

Image credit (also a great tutorial): https://docs.opencv.org/4.x/d1/dee/tutorial_introduction_to_pca.html



Orientation Detection in Code: `b_pca.py` (Line 37-42)

■ Step 1: Preparing the data

■ What this does:

■ **pts** is the contour (the outline points of your object)

```
sz = len(pts)
data_pts = np.empty((sz, 2), dtype=np.float64)
# print(data_pts)
for i in range(data_pts.shape[0]):
    data_pts[i,0] = pts[i,0,0]
    data_pts[i,1] = pts[i,0,1]
```

■ We're converting the contour into a format PCA can understand

■ Creating an empty array to store X and Y coordinates

■ Loop through each point and extract its X position (**pts[i,0,0]**) and Y position (**pts[i,0,1]**)

■ Think of it as: organizing all the dots that make up your object's outline

Orientation Detection in Code: `b_pca.py` (Line 45-46)

■ Step 2: Running PCA

■ What this does:

- `cv.PCACompute2` is OpenCV's PCA function
- `data_pts` is our input (all the outline points)
- `mean` is initially empty
- But PCA will fill it with the center point

```
# Perform PCA analysis
mean = np.empty((0))
mean, eigenvectors, eigenvalues = cv.PCACompute2(data_pts, mean)
```

Orientation Detection in Code: `b_pca.py` (Line 45-46)

■ Step 2: Running PCA

■ What we get back:

- **mean**: The center point (centroid) of the object
- **eigenvectors**: The directions of the principal axes (which way the object points)
- **eigenvalues**: How much the object varies in each direction (length/width information)

```
# Perform PCA analysis  
mean = np.empty((0))  
mean, eigenvectors, eigenvalues = cv.PCACompute2(data_pts, mean)
```

Orientation Detection in Code: b_pca.py (Line 49)

```
# Store the center of the object  
centre = (int(mean[0,0]), int(mean[0,1]))
```

■ Part 3: Finding the center

■ What this does:

- `mean[0,0]` is the X coordinate of the center
- `mean[0,1]` is the Y coordinate of the center
- `int()` converts from decimal to whole number (**pixels must be integers**)
- **centre** is the middle of your object
 - This is where we'll draw from when showing the orientation axes

Orientation Detection in Code: `b_pca.py` (Line 54-56)

```
cv.circle(img, centre, 3, (255, 0, 255), 2)
p1 = (centre[0] + 0.02 * eigenvectors[0,0] * eigenvalues[0,0], centre[1] + 0.02 *
eigenvectors[0,1] * eigenvalues[0,0])
p2 = (centre[0] - 0.02 * eigenvectors[1,0] * eigenvalues[1,0], centre[1] - 0.02 *
eigenvectors[1,1] * eigenvalues[1,0])
```

■ Part 4: Calculating the axis endpoints

■ What this does:

- **p1** is the endpoint of the major axis (longest direction)
- **p2** is the endpoint of the minor axis (perpendicular direction)
- **eigenvectors[0,0]** and **eigenvectors[0,1]** give the direction (X and Y components), **eigenvalues[0,0]** gives the length/scale

Orientation Detection in Code: `b_pca.py` (Line 54-56)

```
cv.circle(img, centre, 3, (255, 0, 255), 2)
p1 = (centre[0] + 0.02 * eigenvectors[0,0] * eigenvalues[0,0], centre[1] + 0.02 *
eigenvectors[0,1] * eigenvalues[0,0])
p2 = (centre[0] - 0.02 * eigenvectors[1,0] * eigenvalues[1,0], centre[1] - 0.02 *
eigenvectors[1,1] * eigenvalues[1,0])
```

■ Part 4: Calculating the axis endpoints

■ What this does:

■ `0.02` is a scaling factor (makes the line visible but not too long)

■ We're calculating: $\text{center} + (\text{direction} \times \text{length} \times \text{scale})$

■ Think of it as: Start at the center, move in the direction the object points, stop at a visible distance

Orientation Detection in Code: `b_pca.py` (Line 57-58)

■ Step 5: Drawing the Axis

■ What this does:

```
drawAxis(img, centre, p1, (0, 255, 0), 1)  
drawAxis(img, centre, p2, (255, 255, 0), 5)
```

■ **drawAxis** is a custom function defined earlier in the code

■ Draws a line from **centre** to **p1** (major axis)

■ **(0, 255, 0)** is the color in BGR format = GREEN

■ **1** is the scale parameter for line thickness

■ Second line draws from **centre** to **p2** (minor axis)

■ **(255, 255, 0)** is BGR = CYAN (which can look blue on the screen)

■ **5** is a thicker line for the minor axis

■ The different colors are set to help you distinguish the two axes

Orientation Detection in Code: `b_pca.py` (Line 60-61)

```
angle = atan2(eigenvectors[0,1], eigenvectors[0,0]) # orientation in radians
angle_in_degrees = -int(np.rad2deg(angle)) * -1 - 90
```

■ Step 6: Calculating the angle

■ What this does:

- `atan2()` calculates the angle from the X and Y components, but **Y comes first** (a mathematical convention)
- `eigenvectors[0,1]` is Y component of major axis
- `eigenvectors[0,0]` is X component of major axis
- `atan2` gives us the angle in radians (mathematical unit for angles)

Orientation Detection in Code: `b_pca.py` (Line 60-61)

```
angle = atan2(eigenvectors[0,1], eigenvectors[0,0]) # orientation in radians
angle_in_degrees = -int(np.rad2deg(angle)) * -1 - 90
```

■ Step 6: Calculating the angle

■ Second line converts and adjusts:

- `np.rad2deg(angle)` converts radians to degrees (easier to understand)
- The `-1 * -1 - 90` adjusts for coordinate system differences
- `int()` rounds to whole number
- Result: angle like "45 degrees" or "-30 degrees"

Orientation Detection in Code: `b_pca.py` (Line 66-68)

```
# Label with the rotation angle
label = "  Rotation Angle: " + str(angle_in_degrees) + " degrees"
textbox = cv.rectangle(img, (centre[0], centre[1]-25), (centre[0] + 250, centre[1]
+ 10), (255,255,255), -1)
cv.putText(img, label, (centre[0], centre[1]), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,
0), 1, cv.LINE_AA)
```

■ Step 7: Displaying the angle

■ What this does:

- `str(angle_in_degrees)` converts number to text
- Creates a label like "Rotation Angle: 45 degrees"

Orientation Detection in Code: `b_pca.py` (Line 66-68)

■ Step 7: Displaying the angle

■ What this does:

- `cv.rectangle` draws a white background box for the text
- `(centre[0], centre[1]-25)` is top-left corner
- `(centre[0] + 250, centre[1] + 10)` is bottom-right corner
- `(255,255,255)` is white color
- `-1` means fill the rectangle (not just outline)

```
# Label with the rotation angle
label = "  Rotation Angle: " + str(angle_in_degrees) + " degrees"
textbox = cv.rectangle(img, (centre[0], centre[1]-25), (centre[0] + 250, centre[1]
+ 10), (255,255,255), -1)
cv.putText(img, label, (centre[0], centre[1]), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,
0), 1, cv.LINE_AA)
```

Orientation Detection in Code: `b_pca.py` (Line 66-68)

■ Step 7: Displaying the angle

■ What this does:

- `cv.putText` writes the text on top
- `(centre[0], centre[1])` is where to start writing
- `0.5` is text size
- `(0,0,0)` is black text color
- Makes the angle readable on screen!

```
# Label with the rotation angle
label = "  Rotation Angle: " + str(angle_in_degrees) + " degrees"
textbox = cv.rectangle(img, (centre[0], centre[1]-25), (centre[0] + 250, centre[1]
+ 10), (255,255,255), -1)
cv.putText(img, label, (centre[0], centre[1]), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,
0), 1, cv.LINE_AA)
```

Let's run: `b_pca.py`

■ Let's run it and see!

■ Go to your Terminal

■ `python b_pca.py`

■ What the code does:

1. Loads an image
2. Converts to grayscale
3. Applies binary threshold
4. Finds contours
5. For each contour, uses PCA to find orientation
6. Draws axes and displays rotation angle

Let's run: `b_pca.py`

- Try:
- What happens if you use your own image with clear objects?
- Observe how the axes align with object orientation

Practical Applications of Edge & Orientation Detection

■ Lane detection (self-driving cars):

- Canny edge detection finds lane markings
- Orientation helps determine road direction

■ Object manipulation (robotics):

- Edge detection finds object boundaries
- Orientation detection tells robot how to grasp

■ Quality control (manufacturing):

- Edge detection finds product defects
- Orientation ensures proper alignment

■ Augmented reality:

- Edge detection finds markers
- Orientation calculates camera pose



Kernels and Convolution Operations



Kernels: The Tool that Makes Edge Detection Work

- A kernel is a small grid (matrix) that slides across your image, kind of like a stamp or a brush tool in Photoshop.
- It's also called a filter or mask
- The kernel is a tiny window that looks at a pixel and its neighbors
- It slides across every pixel in the image
- At each position, it does a calculation based on nearby pixels

Kernels: The Tool that Makes Edge Detection Work

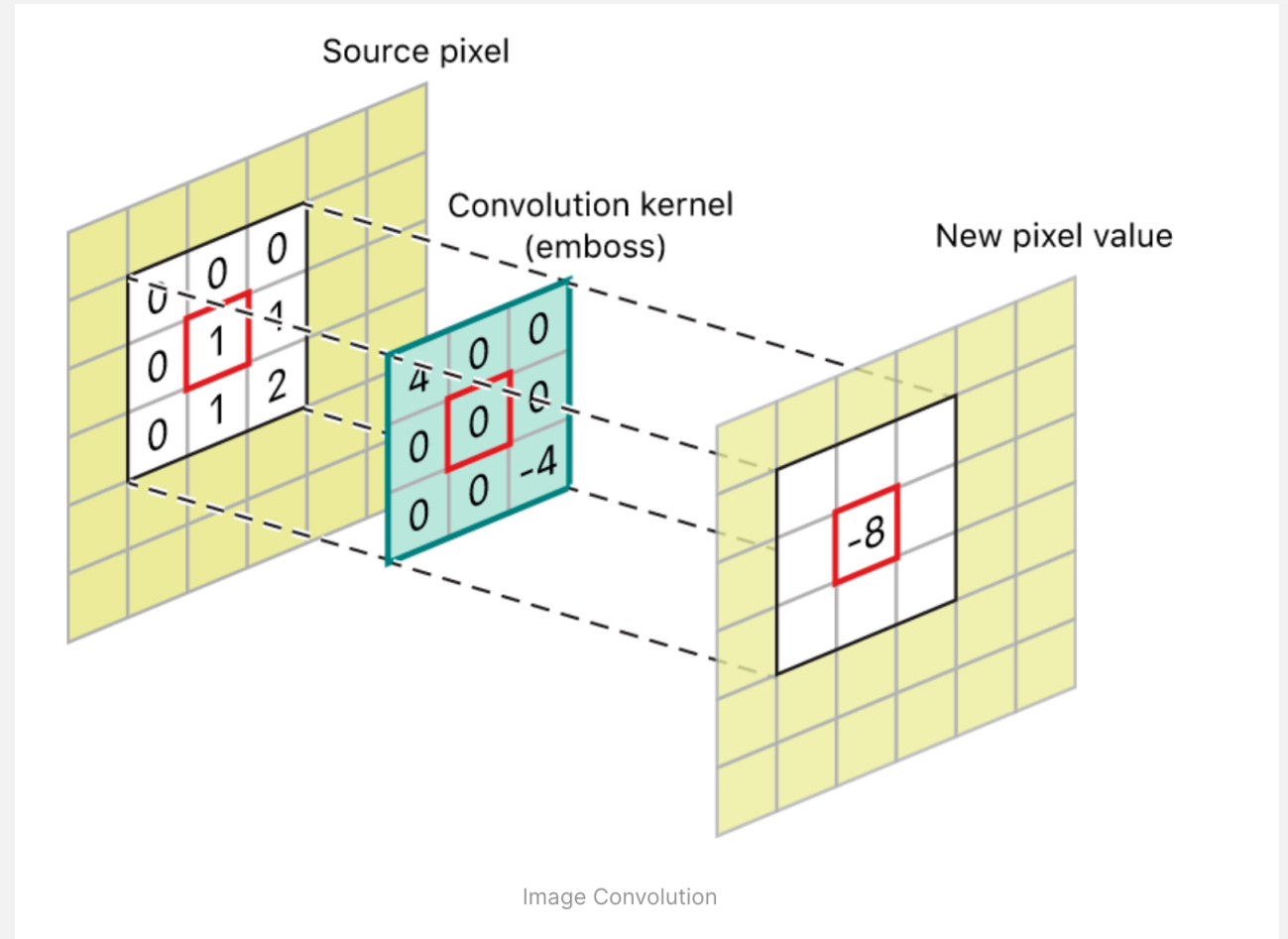


Image credit (also a great tutorial): <https://digitalbunker.dev/understanding-gaussian-blurs/>

Kernels: Size Matters

- Small kernel (3×3): Affects tiny details, detects fine edges

- Subtle changes

- Preserves detail

- Faster processing

- Large kernel (7×7): Affects larger areas, detects bigger features

- Dramatic changes

- Removes more detail

- Slower processing

Kernels: The Tool that Makes Edge Detection Work

■ Shape matters:

- Square kernel: General purpose
- Cross kernel: Horizontal/vertical emphasis
- Circle kernel: Radial effects

■ The kernel is the "brush size" of your image processing tool

■ Experiment with different sizes to find the right one for your task!

■ Reference: <https://www.geeksforgeeks.org/image-filtering-using-convolution-in-opencv/>

What is Convolution?

- The **operation** that makes kernels work
- Convolution is the mathematical operation where:
 1. Place kernel over a section of the image
 2. Multiply each kernel value x corresponding pixel value
 3. Sum all the results
 4. Put that sum in the output image
 5. Slide kernel one pixel over, repeat the process

What is Convolution?

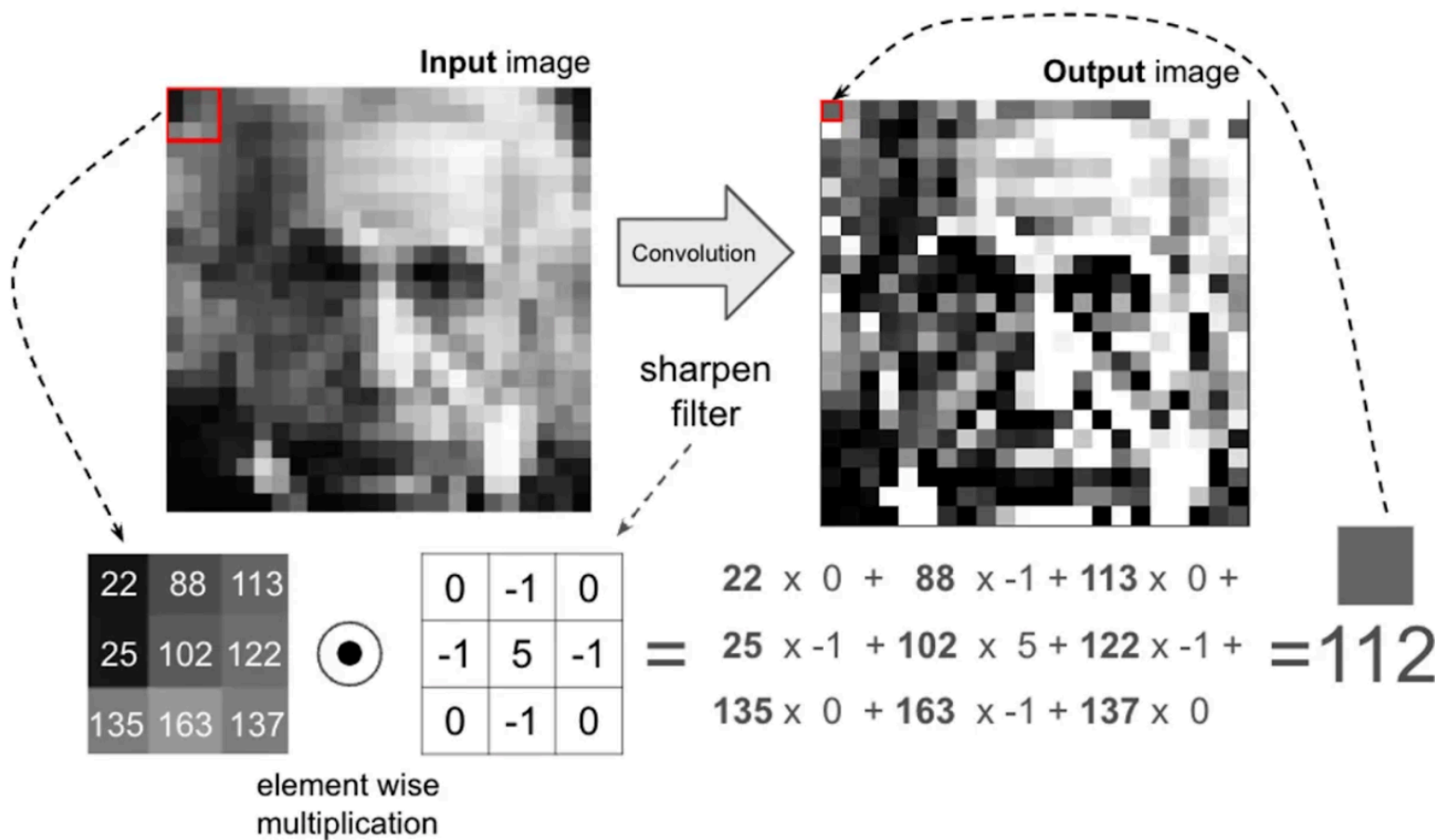


Figure 1: Convolution Kernel Example

You can experiment with the [Convolution Kernels tool](https://muneebbsa.medium.com/deep-learning-101-lesson-20-convolution-kernels-40641dda695d)

Image credit (also a great tutorial): <https://muneebbsa.medium.com/deep-learning-101-lesson-20-convolution-kernels-40641dda695d>

Types of Kernels



Types of Kernels: Blur/Smoothing Kernels

- Different kernel values created different effects
- **Blur/smoothing kernels:**
 - All values positive
 - More weight in the center
 - Smooth, natural blur
 - Better preserves edges
 - Example: Gaussian blur, which you saw in Week 2

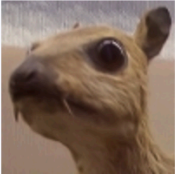
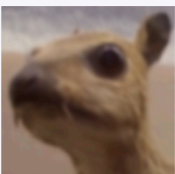
Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

Image credit: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

Types of Kernels: Sharpening Kernels

- Emphasize differences between pixels
- Center pixel gets emphasized (x5 in this case)
- Result: differences are amplified
- Edges and details become more pronounced

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	

Image credit: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

Types of Kernels: Edge Detection Kernels

- Highlight rapid changes in pixel values
- Remember we talked about how edges are places where brightness changes quickly?
- Big differences $\rightarrow$ big numbers $\rightarrow$ likely an edge
- The top checks the center against the four neighbors (up/down/left/right) vs. the second one checks the center against all 8 neighbors $\rightarrow$ The second finds edges more strongly and in all directions

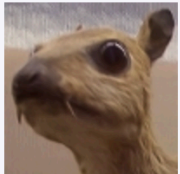
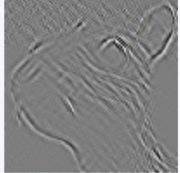
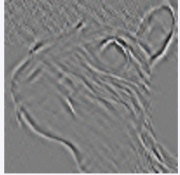
Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

Image credit: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

Why are Kernels Important?

■ For traditional computer vision:

- Foundation of many CV operations
- Fast and efficient
- Mathematically well-understood

■ For machine learning:

- Convolutional Neural Networks (CNNs) learn their own kernels
- Instead of hand-designing kernels*, ML finds optimal ones
- This is why CNNs are so powerful for image analysis

*Hand-designed kernels: people choose the kernel values based on mathematical ideas about the image operation they want (smoothing, sharpening, detecting edges, etc.), not by fitting them to data

Last Week, We Talked About Limitations of Traditional CV

■ Challenge 1: Manual Parameter Tuning

- Every environment is different
- Different lighting → different thresholds
- Different objects → different parameters
- You could spend hours tweaking parameters 😞

■ Challenge 2: Lighting Changes

- Morning vs. evening: same object, different appearance
- Parameters that worked at 9am fail at 5pm

Last Week, We Talked About Limitations of Traditional CV

■ Challenge 2: Lighting Changes

- Shadows detected as objects
- False positives everywhere

■ Challenge 3: Complex Patterns

- Traditional CV struggles with complex, variable objects
- Hard to write rules for every possible scenario

■ This is where ML comes in

Break (15 minutes) ❤️



Introduction to Machine Learning



What is Machine Learning?

- Machine learning is a **subfield** of **artificial intelligence**, which is broadly defined as the capability of a machine to *imitate* intelligent human behavior. Artificial intelligence systems are used to perform complex tasks in a way that is similar to how humans solve problems.
- The goal of AI is to create computer models that exhibit “intelligent behaviors” like humans, according to Boris Katz, a principal research scientist and head of the InfoLab Group at CSAIL. This means machines that can recognize a visual scene, understand a text written in natural language, or perform an action in the physical world.
- Machine learning is one way to use AI. It was defined in the 1950s by AI pioneer Arthur Samuel as “the field of study that gives computers the ability to learn without explicitly being programmed.”

Reference: <https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained>

Traditional Programming vs. ML

■ Traditional Programming:

- Rules + Data → Program → Output
- Example: "If pixel value > 200, it's an edge"
- We define all the rules
- Works great when rules are simple

■ Machine Learning:

- Data + Desired Output → Training → Model → Output
- Example: Show 10,000 images of cats and dogs, ML figures out what makes a cat a cat
- Computer learns the patterns
- Works when rules are too complex to write

How ML Differs from Traditional Programming

■ Traditional CV:

- We designed the edge detection kernel
- We set the threshold values
- We wrote explicit rules for every step

■ Machine Learning:

- We show examples of inputs and outputs
- ML finds the best kernels and parameters automatically
- It learns patterns we might not even notice
- This means:
 - Less manual tuning
 - Better at complex tasks
 - Can adapt to new situations

Machine Learning x Robotics:

■ Perceive complex environments:

- Recognize objects in cluttered scenes
- Handle varying lighting conditions
- Adapt to new objects without reprogramming

■ Learn from experience:

- Improve performance over time

■ Adapt to new tasks

- Transfer knowledge between tasks

■ Handle complexity:

- Problems too complex for hand-written rules
- High-dimensional sensor data
- Real-time decision making



Types of Machine Learning



Type 1: Supervised Learning

- Definition: Learning from labeled data. We tell the algorithm what the correct answer is
- How it works:
 - 1. Provide training data with labels
 - 2. Algorithm learns patterns
 - 3. Can predict labels for new, unseen data

Type 1: Supervised Learning

- For example, the teacher gives the student many past exam questions with correct answers (labelled examples)
- The student studies those pairs and learns how to answer similar future questions
- What the model sees:
 - Input data + correct labels (e.g., images + class names, feature vectors + target values)
 - Task: Learn the mapping from input → label so it can predict answers for new questions.

Type 1: Supervised Learning

- Example: Spam detection

 - Training data: 10,000 emails labeled "spam" or "not spam"

 - ML learns: What patterns make an email spam?

 - Result: Can classify new emails automatically

- Example: Digit recognition (what we'll do next week!)

 - Training data: 60,000 handwritten digits labeled 0-9

 - ML learns: What makes a "7" look like a "7"?

 - Result: Can recognize new handwritten digits

Supervised Learning in Robotics

■ Object recognition:

- Training: Show robot many images labeled "cup", "bottle", "phone"
- Result: Robot can identify these objects in new scenes

■ Gesture classification:

- Training: Show robot videos of hand gestures with labels
- Result: Robot can recognize and respond to gestures

■ Voice commands:

- Training: Audio recordings labeled with commands
- Result: Robot understands spoken instructions

Type 2: Unsupervised Learning

■ Definition: Learning from unlabeled data. The algorithm finds patterns on its own

■ How it works:

1. Provide data without labels
2. Algorithm discovers hidden structure
3. Groups similar items together

Type 2: Unsupervised Learning

- For example, the student gets many exam papers but no answer key. You must group similar papers together, find patterns, or summarize themes without being told what is “correct.”
- What the model sees: Unlabelled data only.
- Task: Discover structure: clusters, low-dimensional representations, common patterns

Type 2: Unsupervised Learning

- Example: Customer segmentation

- Data: Shopping behavior of 100,000 customers

- ML finds: Natural groups (budget shoppers, luxury buyers, etc.)

- No one told it these groups existed

- Example: Anomaly detection

- Data: Normal robot sensor readings

- ML learns: What "normal" looks like

- Can detect when something is wrong

Unsupervised Learning in Robotics

■ Clustering similar objects:

- Robot encounters many objects
- Groups them by similarity without being told what they are

■ Discovering environments:

- Robot explores a space
- Automatically identifies rooms, obstacles, pathway

■ Anomaly detection:

- Robot learns normal operation patterns
- Detects when something is wrong

Type 3: Semi-Supervised Learning

- Definition: Learning from a small amount of labeled data + lots of unlabeled data

- Why useful:

 - Labeling data is expensive and time-consuming

 - Unlabeled data is often abundant

 - Combines benefits of supervised and unsupervised learning

- Example:

 - 100 labeled images of objects

 - 10,000 unlabeled images

 - Use the unlabeled data to improve the model

- Growing importance in robotics where collecting labels is difficult

Type 3: Semi-Supervised Learning

- The teacher gives some papers with answers and many without. The student uses the few labelled examples to guide their interpretation of the many unlabelled one
- What the model sees: A small set of labelled data + a large set of unlabelled data
- Task: Improve performance by leveraging both labelled and unlabelled data
- Examples: Teaching a robot to walk, training a game-playing agent

Type 4: Reinforcement Learning

■ Definition: Learning through trial and error with rewards and punishments

■ How it works:

1. Agent (robot) takes actions in environment
2. Gets rewards for good actions, penalties for bad ones
3. Learns to maximize total reward over time

Type 4: Reinforcement Learning

- For example, the student takes many mock tests and gets feedback as rewards (pass/fail, score) after each attempt. They adjust their strategy over time to maximize long-term success (e.g., which study tactics lead to higher scores)
- What the model sees: An environment, states, actions, and reward signals. There is no explicit correct label for each state, only feedback after actions
- Task: Learn a policy (what action to take in a situation) that maximizes cumulative reward

Type 4: Reinforcement Learning

- Example: Game playing
 - AlphaGo learning to play Go
 - Agent tries moves, learns which ones lead to winning
- Example: Robot locomotion
 - Robot tries different walking patterns
 - Gets reward for moving forward, penalty for falling
 - Learns to walk effectively

Reinforcement Learning in Robotics

■ Robot navigation:

- Trial and error to find optimal paths
- Reward: Reaching goal quickly
- Penalty: Hitting obstacles

■ Manipulation tasks:

- Learning to grasp and place objects
- Reward: Successful task completion
- Penalty: Dropping objects

■ Adaptive control:

- Robot learns to handle changing conditions
- Self-improves through experience
- This can be very powerful but requires lots of training time

Quick Summary

- Supervised = learn from questions and answers
- Unsupervised = find patterns without answers
- Semi-supervised = a few answers + many unlabeled examples
- Reinforcement = learn by trial, error and reward

ML in Robotics: Sense-Plan-Act Revisited

Traditional Sense-Plan-Act

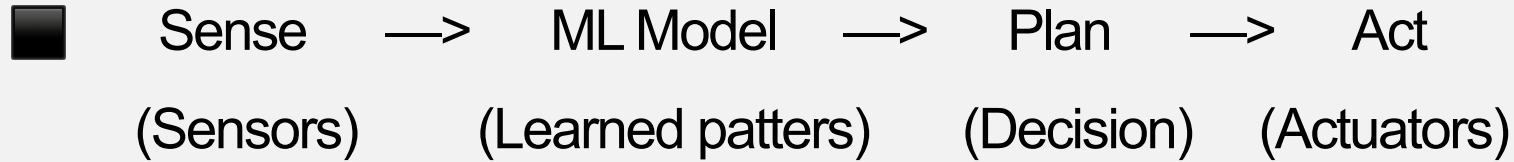


■ In a traditional approach, for example:

1. Sense: Camera detects edge (using Canny)
2. Plan: If edge detected at threshold > 200 , classify as obstacle
3. Act: Turn motors to avoid obstacle

■ All rules are explicitly programmed by us

ML-Enhanced Sense-Plan-Act

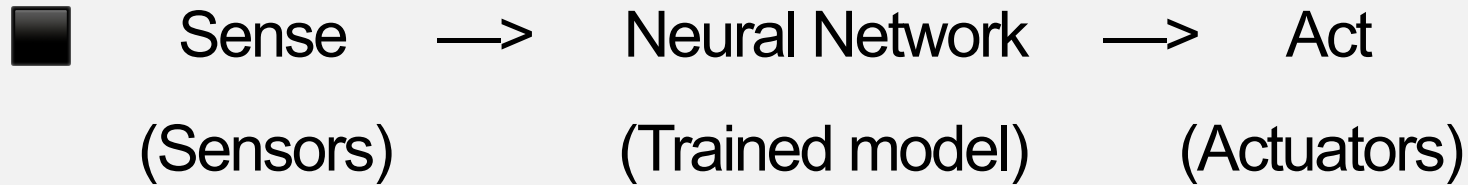


■ In a ML approach, for example:

1. Sense: Camera captures image
2. ML Model: neural network recognizes **obstacles** vs. **safe path**
3. Plan: Decide navigation strategy
4. Act: Move accordingly

■ The recognition part is learned from data

End-to-End Learning: Learned Policy



- In end-to-end approaches, you train a single model that maps raw sensor inputs (e.g., camera images, lidar, proprioception) directly to control outputs (e.g., motor commands, steering, torques), rather than splitting the system into separate perception, planning, and control modules.

End-to-End Learning: Learned Policy

- The decision-making is encoded in the neural network:
- No explicit planning steps
- Camera frames map directly to actions
- Example: NVIDIA's PilotNet takes camera images → steering commands



Figure 5. Screenshot of the Augmented Resimulator simulation platform used to test PilotNet.

Image credit and please read this before Week 4: <https://developer.nvidia.com/blog/researching-and-developing-an-autonomous-vehicle-lane-following-system/>

End-to-End Learning: Learned Policy

■ Benefits:

- Handles complex perceptual inputs end-to-end
- Can learn subtle patterns humans miss

■ Drawbacks:

- Opaque (hard to understand why it makes decisions)
- Can be fragile outside training conditions



Figure 5. Screenshot of the Augmented Resimulator simulation platform used to test PilotNet.

Image credit and please read this before Week 4: <https://developer.nvidia.com/blog/researching-and-developing-an-autonomous-vehicle-lane-following-system/>

End-to-End Learning: Learned Policy

■ Before next week's lecture: please read this NVIDIA blog post shows how end-to-end learning works in practice for autonomous vehicles. Pay special attention to:

- How the system learns from demonstration (human driving)
- The difference between traditional rule-based approaches vs. neural network approach
- How the CNN architecture processes camera images

■ Don't worry if you don't understand all the technical details yet. We will be building our own neural network next week and many concepts will become clearer



Figure 5. Screenshot of the Augmented Resimulator simulation platform used to test PilotNet.

Image credit and please read this before Week 4: <https://developer.nvidia.com/blog/researching-and-developing-an-autonomous-vehicle-lane-following-system/>

Sougwen Chung: Drawing Operations

- Human-robot **collaborative** drawing
- Computer vision (tracking hand movements)
- Neural networks (learning drawing patterns)
- Real-time ML inference (using a trained machine-learning model to make predictions on new data. In other words, running the model to produce outputs such as labels, scores, control commands, embeddings from incoming inputs as they arrive)
- Robotic actuation
- Website



Machine Learning Framework

What is a Framework?

- A framework is:
 - A collection of pre-built tools and functions
 - Makes complex tasks easier
 - Provides structure for your code
 - Handles low-level details so you can focus on high-level logic

What is a Framework?

- For machine learning:
 - Frameworks handle complex math (matrix operations, derivatives)
 - Provide pre-built neural network components
 - Optimize for GPU acceleration
 - Include tools for training and testing

PyTorch

- We will use **PyTorch** in this class
- Why PyTorch?
 - Pythonic: feels natural for Python developers
 - Dynamic computation graphs: more flexible, easier to debug
 - Research-friendly: easy to experiment and prototype
 - Growing adoption: increasingly popular in industry
 - Excellent documentation and community

PyTorch Key Features

■ **Dynamic computation graphs:**

Build neural networks on the fly, easier to debug (you can use print statements)

■ **Tensor operations:**

Similar to NumPy but GPU-accelerated, efficient math operations

■ **Autograd (automatic differentiation):**

Automatically calculates gradients and essential for training neural networks

■ **Rich ecosystem:**

torchvision has pre-built models for computer vision and many research papers release PyTorch Code

TensorFlow and Keras

- TensorFlow was developed by Google Brain. It is production-focused and is industry standard (especially before 2020)
- Keras has high-level API that runs on top of TensorFlow. It's user- and beginner-friendly and great for quick prototyping
- They both have similar goals to PyTorch, build and train neural networks and handle complex ML operations. You can find many tutorials online using Keras/TensorFlow. The code looks similar to PyTorch and concepts can transfer easily

PyTorch vs Keras/TensorFlow: Code Comparison

■ They both define a neural network with two layers. Similar structure, different syntax:

PyTorch style:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

Keras style:

```
from keras import layers, Sequential

model = Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(10, activation='softmax')
])
```

Don't Worry About Framework Choices

- Core ML concepts are the same across frameworks
- Once you learn one, others are easy to pick up
- Many concepts we learn apply to any framework

- **What matters most:**
 - Understanding the concepts (neural networks, training, etc.)
 - Being able to implement ideas
 - Knowing how to experiment and debug



Element 1: In-Class Assignments



What You Have to Produce

- You can find this unit's Assessment Brief on Moodle

- **Element 1: In-class assignments:**

- There will be four in-class assignments to test your understanding of the topics covered in this course (50%)

- This Element 1 brief is on Moodle

- **Element 2: Mini project:**

- A mini project as specified in class (50%)

- The Mini Project brief is on Moodle

Learning Outcomes

- **LO1** Understand how machine learning works in practice (**Knowledge**)
- **LO2** Demonstrate practical skills in computer programming for use in Computer Vision contexts (**Process**)
- **LO3** Use existing frameworks and examples to create computational systems that make use of Computer Vision approaches relevant to robotics (**Realisation**)

What You Have to Produce

- There will be a total of **four in-class assignments** in this class
- The in-assignment briefs will be given in class
- You have until Element Hand-in deadline to turn all four assignments in
 - **Submission Deadline for Element 1 is Monday 26th January 2026 by 2pm**
- For each of the assignment, you will be asked to create at least one customized version of a code example related to computer vision that you will come across during the in-class lab sessions of the Unit

Submission Dates

- Element 1 In-class assignments:

- Submission Deadline: **Monday 26th January 2026 by 2pm**

- Element 2 Mini project:

- Submission Deadline: **Tuesday 27th January 2026 by 2pm**

- Assessment Brief on Moodle

Submission Dates

■ **Adjusted assessment** submission date and time:

- Element 1 In-class assignments: Monday 2nd February 2026 by 2pm
- Element 2 Mini project: Tuesday 3rd February 2026 by 2pm
- Adjusted Assessment (AA) is applicable to students with Individual Support Agreements. To confirm that you intend to use adjusted assessment, please email cci.ug@arts.ac.uk two weeks in advance of the standard deadline.

What You Have to Produce

- The customization in this context may be done by any of the following approaches or any combination of them:
 - Tweaking the setting of the code
 - Changing the code in minor ways
 - Combining the code's function with new or different functions you come across in classes or new examples you find
 - Integrating the code with robotic components and operations

For Element 1 Hand-in

■ Components of your hand-in:

- **Code:** Upload the code for each in-class assignment on your GitHub repository, organized by folders (if there are more than one coding file per assignment)
 - Please make sure files are clearly labeled. For example: assignment1.py
- A short video/ screen recording of you running the code and briefly explained what you did to customize the code for each assignment (less than 5 minutes)
 - You can add annotation on the video but it is not required
 - Adding comments in your code can also be helpful in understanding your code
- Upload the link to Moodle via a PDF

In-class Assignment 1 Brief (Part 1/4 of Element 1 Hand-in)

In-class Assignment #1/4

- Please include the code from this assignment in your GitHub repository for Element 1 hand-in
- Brief: This assignment asks you to create a customized version of one or more code examples from our Week 2 and Week 3 lab files.
- The goal is to demonstrate your understanding of OpenCV and Python by adapting existing code to create something new.

In-class Assignment #1/4

■ You can customize the lab code through any of these approaches (or combinations):

- **Tweaking settings/parameters:** Systematically changing threshold values, kernel sizes, or other parameters and observing effects
- **Changing the code in minor ways:** Modifying what gets displayed, adding new visual feedback, changing colors or text
- **Combining functions:** Integrating two or more OpenCV techniques together (e.g., background subtraction + edge detection)
- **Adding new functionality:** Implementing features we haven't covered but that build on what you've learned
- **Integration with robotics:** (Optional) If you have access to robotic components, integrate the vision code with robot control

In-class Assignment #1/4

- C/Pass: Basic parameter tweaking with minimal exploration, code runs, basic explanation provided
- B Grade: *Systematic* (for example, purposeful testing to experiment with the outcomes) parameter exploration showing genuine experimentation, clear documentation of what works and why, good video explanation demonstrating understanding
- Excellent/A-A+: Combines multiple techniques OR adds significant functionality, shows creative problem-solving beyond basic requirement, well-documented, runs flawlessly, excellent explanation

Combining OpenCV Techniques

- What does it mean to "combine" techniques?
- Key idea: The output of one OpenCV operation can become the input to another operation
- Think of it like a pipeline: Step 1 → produces result → Step 2 uses that result → produces new result → Step 3...
- This is how you build more sophisticated computer vision systems
- Example flow: Camera → Background Subtraction → Edge Detection → Contour Analysis → Draw Results

Understanding Inputs and Outputs

■ Every OpenCV function has:

■ **INPUT:** What it needs (an image, parameters, etc.)

■ **OUTPUT:** What it produces (a processed image, coordinates, numbers, etc.)

■ **`cv.GaussianBlur()`**

■ INPUT: Original image,
kernel size

■ OUTPUT: Blurred
image

■ **`cv.Canny()`**

■ INPUT: Blurred image,
two thresholds

■ OUTPUT: Binary image
(white edges on black
background)

■ **`cv.findContours()`**

■ INPUT: Binary image
(like from Canny)

■ OUTPUT: List of
contours (coordinates)

The Magic of Chaining Operations

■ When the **OUTPUT type** matches the **INPUT type** needed, you can chain them!

■ Example 1:

Blur → Edge Detection

■ `cv.GaussianBlur()` outputs an image

■ `cv.Canny()` needs an image as input

■ ✓ They connect!

■ Example 2:

Edge Detection → Contour Finding

■ `cv.Canny()` outputs a binary image

■ `cv.findContours()` needs a binary image as input

■ ✓ They connect!

■ Example 3:

Background Subtraction → Edge Detection

■ `backSub.apply()` outputs a mask (binary image)

■ `cv.Canny()` needs an image as input

■ ✓ They connect!

How to Approach Combining Techniques

1. Choose your techniques:

What do you want to achieve? Which two (or more) operations would work together?

2. Understand the data flow:

What does Operation A output? Does Operation B accept that type of input?
Draw a diagram if it helps!

3. Start simple:

Get Operation A working by itself first. Get Operation B working by itself first. Then connect them

4. Test and debug:

Display intermediate results (use `cv.imshow()`). Check that data is flowing correctly. One step at a time!

Practical Tips for Combining

1. Display everything:

Show the output of each step in separate windows. This can help you see what's happening at each stage. Example: Window 1 = original, Window 2 = edges, Window 3 = contours on original

2. Check data types:

Some functions need grayscale images. Some need binary images (black and white only). Use `cv.cvtColor()` to convert when needed

3. Copy and paste strategically:

Take working code from lab files. Paste sections in the right order. Connect them by passing outputs as inputs.

4. Comment your code:

Explain what each section does, which will help you (and us!) understand your logic.

Lab Time